

CSC8014 Assessed Coursework

Staff Support Manager

The deadline for this coursework is **14:30 on Thursday, 23rd Feb 2023** and must be submitted on NESS.

The coursework is marked out of **100** and accounts for **100%** of the module mark for CSC8014. All work will be checked for plagiarism. For further information, please see: <https://www.ncl.ac.uk/academic-skills-kit/good-academic-practice/plagiarism/>

1. Aim

The aim of this coursework is for you to practice the design and good practice principles covered in lectures. You will develop interfaces and classes to demonstrate that you have learned and understood the module material, including:

- appropriate overriding of `Object` class methods, including overriding `toString` and providing a static `valueOf` method when appropriate
- design of interface-based hierarchies, programming through interfaces and late binding
- the use of factories to control instantiation of objects, including how to guarantee the instantiation of unique instances
- The use of the Singleton pattern
- the use of defensive programming, including use of immutability and appropriate error handling
- the use of appropriate interfaces and classes from the Collections framework
- appropriate use of Javadocs to document your interfaces and classes
- the use of testing

The coursework is **not** algorithmically challenging. The focus is on good design and good practice.

The coursework is **not** about development of an end-user application. You are developing interfaces and classes that could be used for the development of an application. You should **not** develop a graphical user interface or a command line interface. They are not necessary, and you will be given no credit for doing so.

Note the staff support system specified below is a deliberate simplification. It is not an accurate model of real world University systems. Your solution should correspond to the simplicity of the specification. You risk losing marks if you attempt to provide a more realistic model or provide a solution that is more complicated than necessary.

2. System overview

A University needs a set of interfaces and classes to manage academic staff data. The Department has different types of academic staff. These are **lecturer**, and **researcher** staff. Staff cannot be more than one type. For this coursework, the significant difference between lecturers and researchers is that lecturers can teach on the different modules whereas researchers cannot. Furthermore, researchers can supervise students' projects while lecturers cannot.

The system should maintain records of modules that lecturers teach and students who researcher staff supervise in an academic year. Lecturers can teach up to 40 credits, and researchers can supervise at most 10 students.

When a new staff gets employed, the University needs to be able to issue them a smart card (with a smartcard number) and a staff ID (for login). A staff member can be on either a **permanent** contract or a **fixed** term contract.

3. Tasks

To complete the system outlined in Section 2 you will need to provide interfaces and classes for the functionality described in this section. You must also use Junit testing to unit test your solution.

Task 1

You need to create classes to provide an appropriate hierarchy for staff. Your **StaffManager** class (See Task 4) should create a staff object of the appropriate type when employed. You are provided with the interface **Staff** to start with (See Staff.java in Canvas).

All staff have the following **public** functionality:

- a method to get the staff ID (See Task 2 below).
- a method to get the staff SmartCard (See Task 3 below).
- a method to get the staff type (either Lecturer, or Researcher).
- a method to get the staff Employment Status (either permanent or fixed).
- a method to list the modules that a lecturer is assigned to. A module consists of a name (e.g. Introduction to Software Development), a module code (e.g. CSC8011), a semester (e.g. 1) and the number of credits associated with the module (e.g. 10).
- a method which returns true if the lecturer is currently teaching enough credits (40 credits in both semester) and false otherwise.
- a method to return the list of students who are supervised by a researcher
- a method which returns true if the researcher is currently supervising enough students (10 in total) and false otherwise.

You need to create a separate **Module** class to store module information. You need to create a separate **Name** class to store student first name and last name. You need to make sure that you create these classes with the appropriate methods/variables.

Task 2

You need to create a **StaffID** class. A staff ID has two components - a single letter followed by a three-digit number. For example:

a123

You must provide access to each component and an appropriate string representation of the ID.

Staff ID's are unique. You must guarantee that no two staff have the same ID.

Task 3:

You need to create a **SmartCard** and a **SmartCardNumber** classes. A Smart Card has the staff name (comprising a first and last name – you can use the **Name** class for storing staff first name and last name), the date of birth of the staff, a unique Smart Card number and a date of issue.

The Smart Card number has three components. The first component is the concatenation of the initial of the first name of the staff with the initial of the last name of the staff. The second component is an arbitrary serial number. The third component is the year of issue of the card. For example, the string representation of the Smart Card number for a card issued to John Smith in 2023 would have the form:

JS-10-2023

where the 10 is a serial number that, with the initials and year, guarantees the uniqueness of the card number as a whole.

You must guarantee the uniqueness of smart card numbers.

Your smart card class must provide methods to access the staff name, the staff date of birth, the smart card number and the date of issue of the Card. The smart card should have the following **private** method:

- `setExpiryDate()` ; which sets an expiry date for the card. If the smart card is held by a staff on fixed-term contract, the expiry date is set to the issue date plus two years. If the smart card is held by a staff on permanent contract, the expiry date is set to the issue date plus ten years.

The smart card should have the following **public** method:

- `getExpiryDate()` ; which returns the expiry date of the card.

You should use the [java.util.Date](#) class to represent dates. However, you must not use deprecated methods of the Date class. So, for example, in the test class, you can use [java.util.Calendar](#) to construct dates of birth and dates of issue of Smart Cards. You can assume default time zone and locale.

Task 4:

The **StaffManager** class is the driver class for the university system. It needs to:

- Maintain a record of all staff with their staffID
- Maintain a record of all Students and Modules in the university.

This class need to provide a number of methods. You are provided with **StaffManager.java** class that has the signature of the methods that need to be implemented (Please **DO NOT** change the methods' signature).

- `public Set<Module> readInModules(String path)`
This method should allow modules information to be read from a pre-defined data file (`modules.txt`, where `path` is the path to this file) and stored in a set of modules. The `modules.TXT` file contains one data entry per line with fields separated by a comma e.g. CSC8014, Software Development Advanced Techniques, 2, 10.
- `public Set<Name> readInStudents (String path)`
This method should allow students information to be read from a pre-defined data file (`Students.txt` where `path` is the path to this file) and stored in a set of names. The `Students.TXT` file contains one data entry per line with fields separated by a space e.g. Charlie Chaplin
- `public Staff employStaff(String firstName, String lastName, Date dateOfBirth, String staffType, String employmentStatus)`
This method registers a new staff onto the system and allocates a smart card and a staff ID (see below for additional rules about whether or not a smart card can be issued). On success, this method needs to return a Staff object.
- `public int noOfStaff(String staffType)`
This method returns the number of staff of the specified type (a lecturer or a researcher) that are currently employed.
- `public boolean addData (StaffID id, Set<Module> modules, Set<Name> students)`
This method adds either a set of modules or a set of students to the staff depending on their type. You need to make sure that modules and students are

valid before assigning them to the staff (This can be done by comparing the set against the records of existing students and modules).

- `public Collection<Staff> getAllStaff()`
This method returns all staff that are employed by the university.
- `public void terminateStaff(StaffID id)`
This method removes the staff record associated with the given staff id. In effect, the staff is leaving the University.

When issuing a smart card, the following rules must be observed.

- A staff must be at least 22 years old and at most 67 (retirement age is 68).
- A staff cannot be both a researcher and a lecturer.
- A staff cannot be issued with more than one smartcard (i.e. do not try to deal with lost cards!).

Task 5:

You should provide test cases for your interfaces and classes using Junit testing. You should test the normal case, boundary conditions, and exceptional cases.

4. Notes:

- Make sure that you use of the `Staff.java` and `StaffManager.java` classes provided for you.
- You **must not change** the methods' signature in the `StaffManger` class and in the `Staff` interface. You can add extra methods if needed.
- You **can allow** your methods in the `StaffManager` class to throw an exception where needed.
- If we cannot test your implementation because you have changed the methods' signature, then that corresponding task will score 0.

5. Deliverable (What to submit)

You must submit your work to NESS as a **single** zip file named 'CSC8014_coursework_YourName.zip', where 'YourName' is replaced with your full name. The zip file must contain your solution including test classes (i.e. the implementation of the system and types outlined in Sections 2 and 3).

Your solution should demonstrate:

- the sensible use of Java inheritance mechanisms,
- an understanding of how to use interfaces,
- the ability to handle collections of objects,
- the use of defensive programming, including use of immutability and appropriate error handling,
- an understanding of when and how to override Object methods,
- the implementation of factories and Singleton,
- the ability to implement simple algorithms,
- the ability to write Javadoc comments, and
- the ability to test your code.

5. Marking Scheme

Marks will be allocated for

- Overall structure and implementation of the solution (e.g. classes and their relationships, correct implementation of rules specified in Sections 2 and 3) **(42 Marks)**
- Following good practice guidance: defensive programming, use of immutability, appropriate error handling, appropriate overriding of Object methods, use of object factories where appropriate, use of Singleton where appropriate, etc. **(40 Marks)**
- Evidence of testing by implementation of appropriate test classes that test the normal case, boundary conditions, and exceptional cases **(18 Marks)**

6. Style guidelines

Adopt a consistent style, do not violate naming conventions (e.g. when to use upper/lower case letters in class, method and variable names) and make appropriate use of whitespace (Indentation and other spacing).

7. Further notes

Start early!! This is not a project to hack together during the last 1-2 days before the deadline.

Break the coursework down into separate tasks. Start with the simpler classes first (e.g. **Module**, **Name**, **StaffID**, **SmartCardNumber** and **SmartCard**) but leave the imposition of uniqueness and immutability until we cover these topics in lectures. You can implement the different types of staff before implementing the **StaffManager** class. Unit test classes as you progress through the coursework.

For each class you implement you should consider:

- whether to override Object methods (equals, toString, , etc.) and use valueOf,
- whether to use an interface-based hierarchy, and
- whether the class should be immutable.

You may have to defer parts of the coursework (or the implementation of certain aspects of a class) until we have covered material in lectures. In which case, you can make a start with a simpler solution that can be extended later. For example, we have not covered the Collections framework yet. To make a start, you can just use a single instance of a class.

For any questions, email me at **Rouaa.yassin-kassab@ncl.ac.uk**